

Documentação do Sistema

1. Visão geral

O projeto implementa um sistema agetivo orientado a evidências para a análise de dados de monitoramento de biodiversidade, com foco em comunidades de aves. O objetivo principal da solução é permitir que um usuário faça perguntas em linguagem natural sobre um conjunto de dados e receba respostas fundamentadas em resultados analíticos reproduzíveis, auditados e rastreáveis.

A arquitetura foi projetada para evitar que o modelo de linguagem seja responsável diretamente pelos cálculos analíticos. O LLM atua principalmente na interpretação da solicitação do usuário e na redação da resposta final, enquanto as operações críticas são executadas por ferramentas determinísticas desenvolvidas em Python. Dessa forma, o modelo de linguagem não constitui a fonte primária dos resultados apresentados pelo sistema.

O fluxo geral começa com uma pergunta em linguagem natural enviada pelo usuário. Essa pergunta é encaminhada ao registro de skills, que identifica qual capacidade analítica é compatível com a solicitação. Em seguida, o orchestrator coordena a execução da ferramenta determinística correspondente. O resultado produzido é encapsulado em uma evidência, submetido a uma etapa independente de auditoria e, somente após essa validação, encaminhado ao serviço de LLM para geração da resposta contextualizada. Paralelamente, o sistema registra informações da execução e gera um relatório persistente para garantir rastreabilidade.

A principal característica arquitetural da solução é a separação entre interpretação da pergunta, execução dos cálculos, produção da evidência, auditoria independente, redação da resposta e registro da execução. Essa separação reduz a possibilidade de uma resposta gerada pelo modelo de linguagem substituir, distorcer ou inventar um resultado que deveria ser determinado diretamente pelos dados.

2. Princípio central: Evidence First

O projeto segue uma abordagem denominada Evidence First. Nesse modelo, uma conclusão analítica não é considerada confiável simplesmente porque foi produzida pelo sistema. O resultado passa por uma cadeia composta pelo processamento dos dados, execução de uma análise determinística, geração de um Evidence Claim, auditoria independente e definição do estado da evidência antes de ser utilizado na resposta final.

A unidade central dessa arquitetura é o Evidence Claim. Ele registra informações necessárias para identificar a análise realizada, sua fonte de dados, os parâmetros utilizados, o resultado obtido, o momento da execução, o status da análise, as informações necessárias para reprodução e o estado da auditoria.

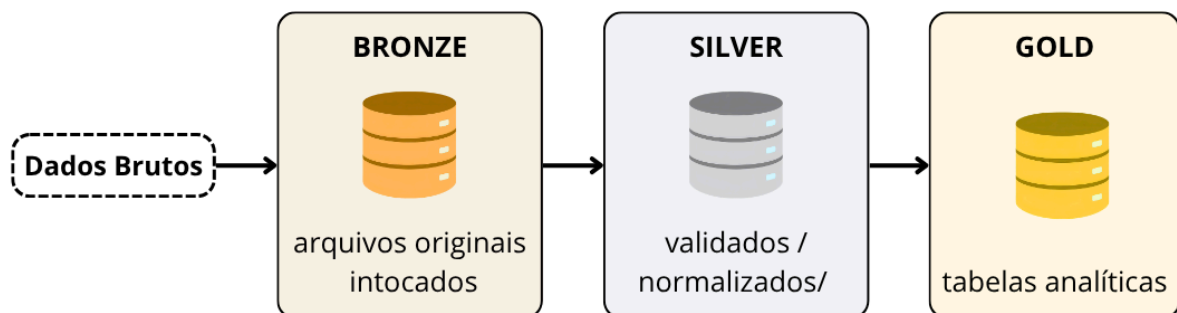
O sistema diferencia explicitamente três conceitos: o resultado calculado, o resultado auditado e a interpretação textual. Um valor pode ter sido calculado corretamente por uma ferramenta, mas ainda precisa passar pela etapa de auditoria antes de ser tratado como uma conclusão analítica verificada. Da mesma forma, a interpretação textual produzida pelo LLM não substitui o resultado calculado.

Essa separação permite manter uma fronteira clara entre a parte determinística e a parte generativa da aplicação. O código determina os resultados, o auditor verifica esses resultados e o modelo de linguagem os comunica ao usuário.

3. Arquitetura de dados

O processamento dos dados segue a arquitetura medalhão de três camadas: Bronze, Silver e Gold.

Arquitetura Medalhão para Engenharia de Dados:



A camada Bronze contém os dados brutos obtidos das fontes originais. Entre os arquivos utilizados estão `event.txt`, `occurrence.txt` e `humboldt ecological inventory.txt`. Nessa etapa, os valores originais são preservados e os arquivos funcionam como referência primária para as etapas posteriores de processamento e rastreabilidade.

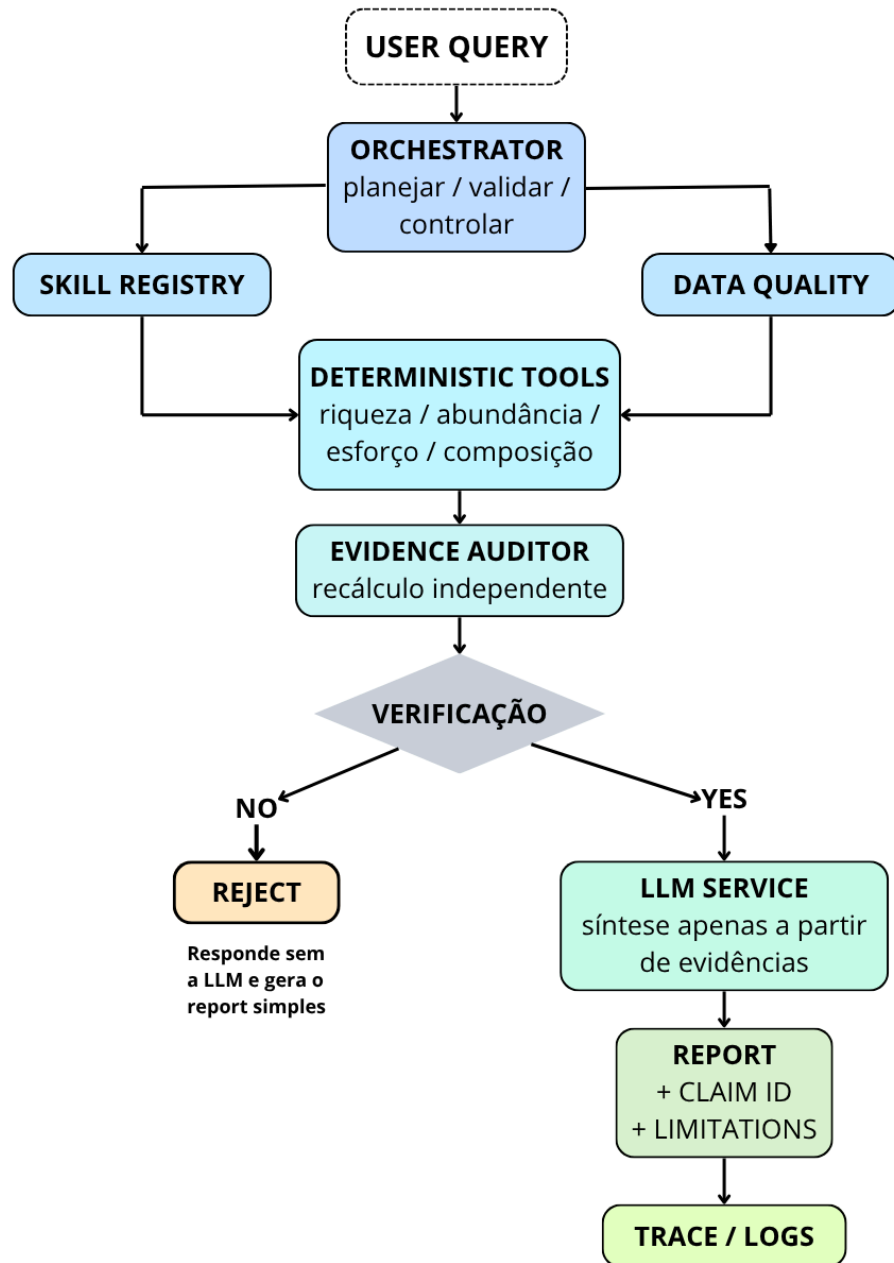
A camada Silver contém os dados processados e estruturados para utilização pelas ferramentas analíticas. Seus principais arquivos são `eventos.csv`, `ocorrencias.csv`, `inventarios.csv`, `perfil_dados.json` e `relatorio_qualidade.json`. Essa camada realiza as transformações estruturais necessárias para que os dados possam ser analisados sem modificar a representação original da fonte. Também é nessa etapa que são identificados problemas relacionados à qualidade dos dados, às relações entre entidades, à estrutura dos registros e às limitações metodológicas.

A camada Gold Analytical contém representações específicas para as análises disponibilizadas pelo sistema. Entre os principais arquivos estão `richness_event.csv`, `richness_month.csv`, `abundance_event.csv`, `composition_event.csv` e `effort_event.csv`. Essas tabelas não representam simplesmente uma cópia da camada Silver, mas estruturas preparadas para responder perguntas analíticas específicas, por exemplo, `richness_month.csv` é utilizado para análise da riqueza observada por mês.

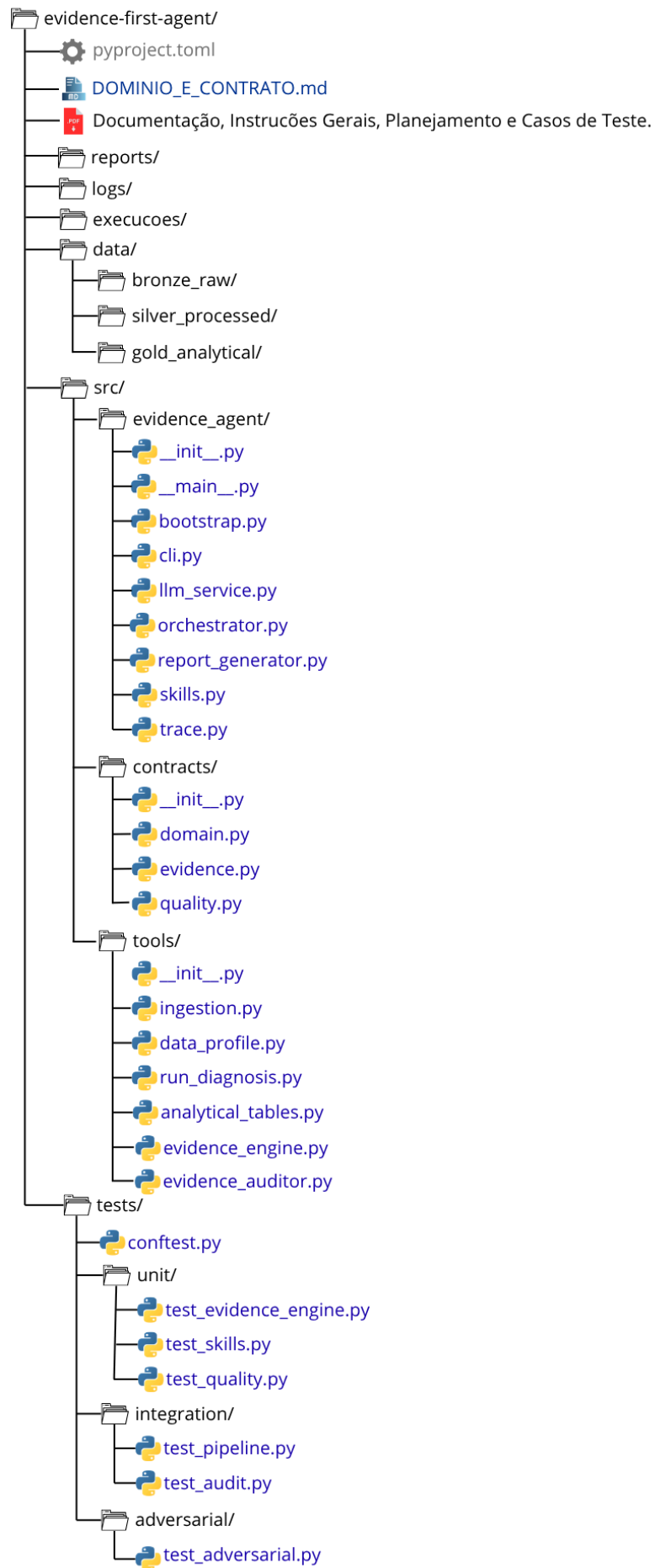
A separação entre Silver e Gold evita que cada ferramenta analítica precise reconstruir toda a estrutura dos dados a cada execução, permitindo que as análises trabalhem sobre estruturas previamente preparadas e mais previsíveis.

4. Arquitetura Geral e Sistema de Pastas e Arquivos

Arquitetura Geral:



Estrutura de Pastas e Arquivos:



5. Contratos do domínio

O diretório `src/evidence_agent/contracts/` contém os contratos utilizados para definir as estruturas esperadas pelo sistema. Os módulos são `domain.py`, `evidence.py` e `quality.py`.

O arquivo `domain.py` define estruturas relacionadas ao domínio da análise e representa formalmente conceitos relevantes para o sistema, como eventos, ocorrências e informações associadas ao monitoramento. O objetivo é estabelecer uma separação entre os dados recebidos e a estrutura esperada internamente pela aplicação, permitindo detectar entradas incompatíveis antes que sejam utilizadas pelas análises.

O arquivo `evidence.py` define as estruturas relacionadas às evidências produzidas pelo sistema. O conceito central é o Evidence Claim, que permite registrar qual análise foi realizada, sobre qual fonte, com quais parâmetros, qual resultado foi obtido e qual é o estado de auditoria associado ao resultado. Essa estrutura estabelece uma relação formal entre uma resposta e o resultado analítico específico que a originou.

O arquivo `quality.py` concentra estruturas relacionadas à avaliação da qualidade dos dados. São considerados aspectos como integridade das referências, duplicidades, estrutura dos registros, compatibilidade dos campos, problemas de tipagem, presença de múltiplos protocolos de amostragem, diferentes representações de esforço e registros de presença sem contagem quantitativa.

6. Módulos do sistema

6.1. Módulo de ingestão

O módulo localizado em `src/evidence_agent/tools/ingestion.py` é responsável por transformar os arquivos da camada Bronze em estruturas processáveis. O processo envolve a leitura dos dados, sua normalização estrutural, o relacionamento entre entidades e a geração das estruturas utilizadas na camada Silver.

6.2. Módulo de perfil dos dados

O módulo `src/evidence_agent/tools/data_profile.py` produz um diagnóstico inicial da base. Ele avalia características como quantidade de eventos, quantidade de ocorrências, quantidade de inventários, referências sem correspondência, duplicidades, problemas de schema, protocolos de amostragem, campos relacionados ao esforço e registros presentes sem contagem quantitativa. O resultado desse processamento é armazenado em `data/silver_processed/perfil_dados.json`.

6.3. Módulo de diagnóstico

O arquivo `src/evidence_agent/tools/run_diagnosis.py` funciona como ponto de preparação do dataset. Sua responsabilidade é coordenar as etapas de ingestão, geração do perfil dos dados, validação e produção do relatório de qualidade. Como

resultado, é produzido o arquivo `data/silver_processed/relatorio_qualidade.json`, que consolida os principais problemas identificados e suas respectivas limitações. Essa etapa permite que a qualidade dos dados seja considerada como parte do processo analítico, em vez de ser tratada como uma preocupação independente e posterior à análise.

6.4. Módulo de tabelas analíticas

O módulo `src/evidence_agent/tools/analytical_tables.py` transforma os dados Silver em estruturas específicas para as análises disponíveis. Sua principal finalidade é separar o processamento geral dos dados da execução das perguntas analíticas. As tabelas produzidas constituem a camada Gold Analytical e funcionam como uma interface entre a preparação dos dados e o Evidence Engine.

6.5. Evidence Engine

O arquivo `src/evidence_agent/tools/evidence_engine.py` é responsável pelo núcleo determinístico das análises. Esse componente executa os cálculos utilizados para produzir os resultados do sistema e não utiliza o LLM para realizar operações analíticas. Atualmente, o Evidence Engine disponibiliza três análises principais: `get_peak_richness_by_month()`, `get_abundance_summary()` e `get_effort_discrepancy_report()`.

6.6. Evidence Auditor

O arquivo `src/evidence_agent/tools/evidence_auditor.py` é responsável por fornecer uma segunda camada de verificação dos resultados. O auditor não simplesmente confia no resultado produzido pelo EvidenceEngine. Ele realiza verificações ou recálculos independentes dos resultados críticos. Dessa forma, existe uma separação entre o componente responsável por produzir um resultado e o componente responsável por verificar esse resultado.

O sistema trabalha com estados que representam o grau de suporte disponível para uma conclusão. Entre os estados conceituais estão `VERIFIED`, `SUPPORTED`, `UNCERTAIN` e `UNSUPPORTED`.

O estado `VERIFIED` representa a situação em que o resultado produzido passou pela verificação independente prevista pelo sistema. Um resultado que ainda não passou pela auditoria não deve ser tratado como uma conclusão analítica verificada. Essa distinção cria uma barreira entre o resultado simplesmente calculado e o resultado que foi efetivamente aceito para comunicação como evidência verificada.

6.7. Skill Registry

O arquivo `src/evidence_agent/skills.py` contém o SkillRegistry, responsável por identificar qual capacidade analítica deve ser utilizada para responder à pergunta do usuário.

As skills atualmente disponíveis são `peak_richness`, `abundance_summary` e `effort_discrepancy`. O registro funciona como uma camada de roteamento entre a linguagem natural utilizada pelo usuário e as ferramentas determinísticas existentes no sistema. Quando nenhuma skill compatível é identificada, a solicitação é considerada fora do escopo analítico implementado.

- **Skill `peak_richness`:** responsável por perguntas relacionadas à maior riqueza observada, ao mês com maior riqueza, ao número de espécies observadas por período e ao pico de riqueza.
- **Skill `abundance_summary`:** responsável por perguntas relacionadas à abundância, à abundância quantificada, aos registros quantitativos, aos registros de presença e à quantidade de indivíduos.
- **Skill `effort_discrepancy`:** responsável por perguntas relacionadas ao esforço amostral, às diferenças de esforço, às unidades utilizadas, às diferentes representações de esforço e à sua comparabilidade.

6.8. Orchestrator

O arquivo `src/evidence_agent/orchestrator.py` contém o `AnalysisOrchestrator`, responsável por coordenar o fluxo completo de uma solicitação. ele conecta os diferentes componentes da arquitetura. Ao receber uma pergunta, ele valida a entrada, identifica a skill apropriada, executa a análise determinística, gera o Evidence Claim, executa a auditoria, verifica o estado da evidência, gera a resposta, cria o relatório e registra a execução.

Cada execução recebe um `execution_id`, que representa uma execução concreta do sistema. Enquanto o `claim_id` identifica uma análise e seus parâmetros, o `execution_id` identifica a execução específica realizada pelo usuário.

6.9. LLM Service e Fallback determinístico

O arquivo `src/evidence_agent/llm_service.py` é responsável pela interface entre o sistema e o modelo de linguagem. O modelo pode ser executado localmente através do Ollama. Entretanto, o LLM não é responsável por calcular riqueza, abundância ou esforço, modificar os dados, validar matematicamente uma conclusão ou substituir o auditor. Sua principal função é transformar uma evidência já produzida pelo sistema. Essa separação pode ser resumida da seguinte forma: o Python calcula, o auditor verifica e o LLM comunica.

Além disso, o sistema possui uma resposta alternativa para situações em que o Ollama ou o pacote Python correspondente não esteja disponível ou quando ocorra um erro na comunicação com o modelo. Nesse cenário, a aplicação não precisa interromper completamente a análise. Ela pode retornar uma resposta baseada diretamente na evidência disponível, mantendo o fluxo determinístico funcional mesmo quando a camada generativa não estiver disponível.

6.10. Report Generator

O arquivo `src/evidence_agent/report_generator.py` é responsável por transformar os resultados de uma execução em um relatório persistente. Os relatórios são armazenados no diretório `reports/`. Um relatório pode conter a pergunta original, o timestamp da execução, a análise realizada, o Claim ID, o estado da evidência, o status da auditoria, a fonte dos dados, o resultado obtido, o resultado recalculado pelo auditor e as informações necessárias para rastreabilidade. Permitindo consultar posteriormente o resultado de uma execução sem depender da memória da conversa ou da sessão em que a análise foi realizada.

6.11. Execution Tracer

O arquivo `src/evidence_agent/trace.py` é responsável por registrar informações resumidas das execuções realizadas. Os registros são armazenados em formato JSON Lines no arquivo `logs/execution_trace.jsonl`. Cada execução registra informações como `execution_id`, timestamp, pergunta do usuário, status, skill executada, Claim ID, status da auditoria e arquivo de relatório associado. O uso do formato JSONL permite adicionar novas execuções continuamente sem sobrescrever registros anteriores, criando um histórico simples, estruturado e legível das operações realizadas.

6.12. Bootstrap

O arquivo `src/evidence_agent/bootstrap.py` é responsável por preparar e verificar o ambiente de dados necessário para a execução do sistema. O bootstrap verifica a existência dos arquivos obrigatórios da camada Bronze e também verifica a presença das principais tabelas Gold, sua função é garantir que a estrutura mínima esperada pelo sistema esteja disponível antes da execução das análises.

6.13. CLI

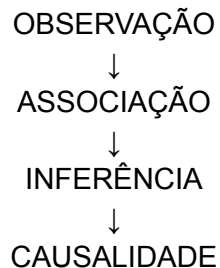
O arquivo `src/evidence_agent/cli.py` fornece a interface de execução do sistema pelo terminal. Por meio da CLI, o usuário pode iniciar a aplicação e enviar uma pergunta diretamente pela linha de comando. A CLI abstrai os detalhes internos da implementação e permite que o usuário interaja com o sistema sem precisar conhecer a organização dos módulos.

7. Fluxo completo de execução

Quando o usuário realiza uma pergunta, o SkillRegistry identifica a capacidade analítica compatível com a pergunta e o orchestrator chama a ferramenta determinística correspondente. O resultado calculado é então encapsulado em um Evidence Claim. Esse claim contém, entre outras informações, seu identificador, a análise executada, a fonte dos dados, os parâmetros e o resultado obtido. Nesse momento, o estado da auditoria ainda pode ser considerado pendente. O EvidenceAuditor executa então a verificação independente do resultado. Se os resultados forem consistentes, o estado de auditoria pode ser definido como `VERIFIED`. Caso contrário, o sistema não deve apresentar o resultado

como uma conclusão analítica verificada. Depois da validação, o resultado é encaminhado ao LLMService, que recebe a evidência produzida pelo sistema e transforma o resultado em uma resposta compreensível para o usuário. Paralelamente, o ReportGenerator produz um relatório persistente e o ExecutionTracer registra a execução. Uma das propriedades centrais do sistema é permitir a rastreabilidade e reconstrução de uma resposta.

A interpretação das análises segue uma hierarquia conceitual:



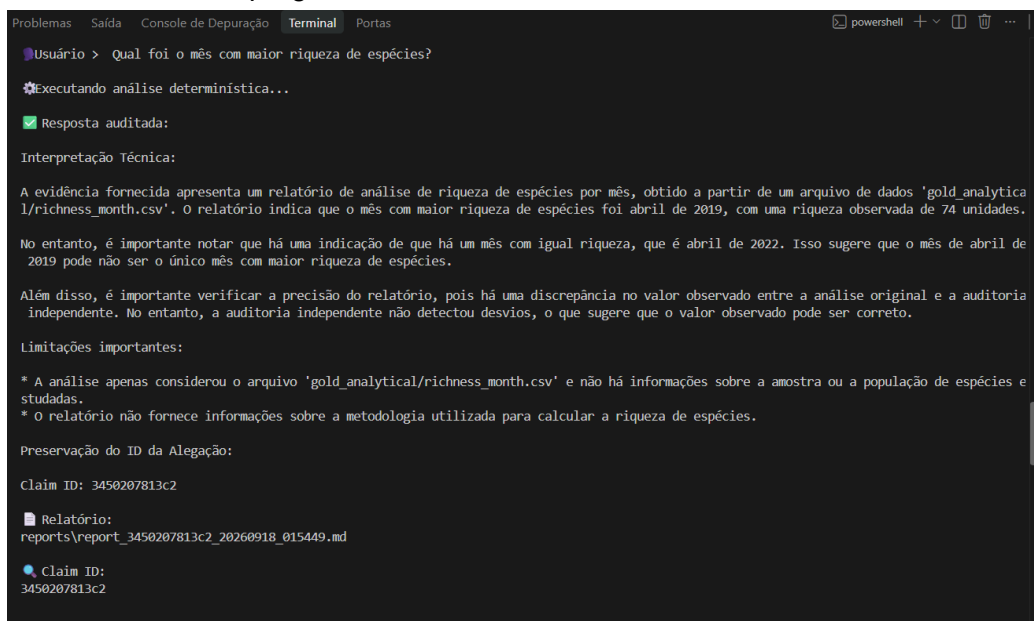
A **reprodutibilidade** do sistema é garantida principalmente pela separação entre componentes determinísticos e generativos, pois o fluxo de informações permite identificar o caminho percorrido por uma análise e reproduzir ou verificar os resultados obtidos.

8. Dependências e modelo de execução

As dependências principais do projeto são definidas no arquivo `pyproject.toml`. Entre as dependências essenciais estão `pandas` e `pydantic`. A integração com o modelo de linguagem utiliza o pacote `ollama` que deve ser instalado previamente no computador e chamado no python antes de do projeto ser executado. Toda as instruções de execução e requisitos podem ser verificados no arquivo “Instruções Gerais.pdf”

9. Exemplos de resultados esperados

- Resultado de uma pergunta:



```
Problemas Saída Console de Depuração Terminal Portas
Usuário > Qual foi o mês com maior riqueza de espécies?
Executando análise determinística...
Resposta auditada:
Interpretação Técnica:
A evidência fornecida apresenta um relatório de análise de riqueza de espécies por mês, obtido a partir de um arquivo de dados 'gold_analytica/1/richness_month.csv'. O relatório indica que o mês com maior riqueza de espécies foi abril de 2019, com uma riqueza observada de 74 unidades. No entanto, é importante notar que há uma indicação de que há um mês com igual riqueza, que é abril de 2022. Isso sugere que o mês de abril de 2019 pode não ser o único mês com maior riqueza de espécies. Além disso, é importante verificar a precisão do relatório, pois há uma discrepância no valor observado entre a análise original e a auditoria independente. No entanto, a auditoria independente não detectou desvios, o que sugere que o valor observado pode ser correto.
Limitações importantes:
* A análise apenas considerou o arquivo 'gold_analytica/1/richness_month.csv' e não há informações sobre a amostra ou a população de espécies estudadas.
* O relatório não fornece informações sobre a metodologia utilizada para calcular a riqueza de espécies.
Preservação do ID da Alegação:
Claim ID: 3450207813c2
Relatório:
reports/report_3450207813c2_20260918_015449.md
Claim ID:
3450207813c2
```

- Exemplo de relatório gerado

Biodiversity Analysis Report

Question: "O que o conjunto de dados mostra sobre a abundância quantificada?"

Issued: 2026-09-18 01:28:35 UTC

Analysis: `abundance_summary`

Evidence Claim: `eb1dcd875a16`

Evidence State: `VERIFIED`

Audit Status: `VERIFIED`

1. Executive Conclusion

O resultado analítico não foi verificado de forma independente. Nenhuma conclusão substantiva foi fornecida.

2. Evidence

Data Source: `gold_analytical/abundance_event.csv`

Reproducible: `True`

Calculated Result

```
{
  "total_quantified_abundance": 46401.0,
  "total_qualitative_records": 2391,
  "total_records_analyzed": 15033,
  "qualitative_ratio": 0.1591
}
```

Independently Recalculated Result

```
{
  "total_quantified_abundance": 46401.0,
  "total_qualitative_records": 2391,
  "total_records_analyzed": 15033
}
```

Discrepancy Detected: `False`

3. Audit

The result above was independently recalculated by the evidence auditor.

Audit Status: `VERIFIED`

- Exemplo de resultado de um diagnóstico dos dados feito automaticamente pelo sistema:

```
=====
RESUMO DO DIAGNÓSTICO
=====
Eventos:      814
Ocorrências:  96,246
Inventários:  980
Problemas:    4
Status:       VÁLIDO
=====
```

- Exemplo de manifesto.json da pasta de execuções:

```
execucoes > 20260917_031603_01bd625b > {} manifesto.json > ...
1  {
2    "execution_id": "20260917_031603_01bd625b",
3    "question": "Como os dados estão estruturados e quais problemas de qualidade podem afetar a análise da comunidade de a
4    "input_files": [
5      "data\\bronze_raw\\event.txt",
6      "data\\bronze_raw\\occurrence.txt",
7      "data\\bronze_raw\\humboldtecollogicalinventory.txt"
8    ],
9    "output_files": [
10     "data\\silver_processed\\eventos.csv",
11     "data\\silver_processed\\ocorrencias.csv",
12     "data\\silver_processed\\inventarios.csv"
13   ],
14   "quality_report": "execucoes\\20260917_031603_01bd625b\\relatorio_qualidade.json",
15   "status": "VALIDO"
16 }
```